LaTeX's hook management for files*

Frank Mittelbach, Phelype Oleinik, LaTeX Project Team

November 3, 2026

Contents

1 Introduction **1**
1.1 Provided hooks . . . 1
1.2 General hooks for file reading . . . 1
1.3 Hooks for package and class files . . . 3
1.4 Hooks for `\include` files . . . 4
1.5 High-level interfaces for LaTeX . . . 5
1.6 Kernel, class, and package interfaces for LaTeX . . . 5
1.7 A sample package for structuring the log output . . . 5

Index **7**

1 Introduction

1.1 Provided hooks

The code offers a number of hooks into which packages (or the user) can add code to support different use cases. Many hooks are offered as pairs (i.e., the second hook is reversed. Also important to know is that these pairs are properly nested with respect to other pairs of hooks.

There are hooks that are executed for all files of a certain type (if they contain code), e.g., for all "include files" or all "packages", and there are also hooks that are specific to a single file, e.g., do something after the package `foo.sty` has been loaded.

1.2 General hooks for file reading

There are four hooks that are called for each file that is read using document-level commands such as `\input`, `\include`, `\usepackage`, etc. They are not called for files read using internal low-level methods, such as `\@input` or `\openin`.

*This code has version v1.0q dated 2026-11-01, © LaTeX Project.

file/before
file/.../before
file/.../after
file/after

These are:

file/before, file/⟨*file-name*⟩/before These hooks are executed in that order just before the file is loaded for reading. The code of the first hook is used with every file, while the second is executed only for the file with matching ⟨*file-name*⟩ allowing you to specify code that only applies to one file.

file/⟨*file-name*⟩/after, file/after These hooks are executed after the file with name ⟨*file-name*⟩ has been fully consumed. The order is swapped (the specific one comes first) so that the `/before` and `/after` hooks nest properly, which is important if any of them involve grouping (e.g., contain environments, for example). Furthermore both hooks are reversed hooks to support correct nesting of different packages adding code to both `/before` and `/after` hooks.

So the overall sequence of hook processing for any file read through the user interface commands of LaTeX is:

```
\UseHook{file/before}
\UseHook{file/⟨file name⟩/before}
   ⟨file contents⟩
\UseHook{file/⟨file name⟩/after}
\UseHook{file/after}
```

The file hooks only refer to the file by its name and extension, so the ⟨*file name*⟩ should be the file name as it is on the filesystem with extension (if any) and without paths. Different from `\input` and similar commands, the `.tex` extension is not assumed in hook ⟨*file name*⟩, so `.tex` files must be specified with their extension to be recognized. Files within subfolders should also be addressed by their name and extension only.

Extensionless files also work, and should then be given without extension. Note however that TeX prioritizes `.tex` files, so if two files `foo` and `foo.tex` exist in the search path, only the latter will be seen.

When a file is input, the ⟨*file name*⟩ is available in `\CurrentFile`, which is then used when accessing the `file/`⟨*file name*⟩`/before` and `file/`⟨*file name*⟩`/after`.

\CurrentFile

The name of the file about to be read (or just finished) is available to the hooks through `\CurrentFile` (there is no `expl3` name for it for now). The file is always provided with its extension, i.e., how it appears on your hard drive, but without any specified path to it. For example, `\input{sample}` and `\input{app/sample.tex}` would both have `\CurrentFile` being `sample.tex`.

\CurrentFilePath

The path to the current file (complement to `\CurrentFile`) is available in `\CurrentFilePath` if needed. The paths returned in `\CurrentFilePath` are only user paths, given through `\input@path` (or `expl3`'s equivalent `\l_file_search_path_seq`) or by directly typing in the path in the `\input` command or equivalent. Files located by `kpsewhich` get the path added internally by the TeX implementation, so at the macro level it looks as if the file were in the current folder, so the path in `\CurrentFilePath` is empty in these cases (package and class files, mostly).

\CurrentFileUsed
\CurrentFilePathUsed

In normal circumstances these are identical to \CurrentFile and \CurrentFilePath. They will differ when a file substitution has occurred for \CurrentFile. In that case, \CurrentFileUsed and \CurrentFilePathUsed will hold the actual file name and path loaded by LaTeX, while \CurrentFile and \CurrentFilePath will hold the names that were *asked for*. Unless doing very specific work on the file being read, \CurrentFile and \CurrentFilePath should be enough.

1.3 Hooks for package and class files

Commands to load package and class files (e.g., \usepackage, \RequirePackage, \LoadPackageWithOptions, etc.) offer the hooks from section 1.2 when they are used to load a package or class file, e.g., file/array.sty/after would be called after the array package got loaded. But as packages and classes form as special group of files, there are some additional hooks available that only apply when a package or class is loaded.

package/before
package/after
package/.../before
package/.../after
class/before
class/after
class/.../before
class/.../after

These are:

package/before, package/after These hooks are called for each package being loaded.

package/⟨*name*⟩/before, package/⟨*name*⟩/after These hooks are additionally called if the package name is ⟨*name*⟩ (without extension).

class/before, class/after These hooks are called for each class being loaded.

class/⟨*name*⟩/before, class/⟨*name*⟩/after These hooks are additionally called if the class name is ⟨*name*⟩ (without extension).

All /after hooks are implemented as reversed hooks.

The overall sequence of execution for \usepackage and friends is:

```
\UseHook{package/before}
\UseOneTimeHook{package/⟨package name⟩/before}
    \UseHook{file/before}
    \UseHook{file/⟨package name⟩.sty/before}
        ⟨package contents⟩
    \UseHook{file/⟨package name⟩.sty/after}
    \UseHook{file/after}
    code from \AtEndOfPackage if used inside the package
\UseOneTimeHook{package/⟨package name⟩/after}
\UseHook{package/after}
```

and similar for class file loading, except that package/ is replaced by class/ and \AtEndOfPackage by \AtEndOfClass.

If a package or class is not loaded none of the hooks are executed!

All class or package hooks involving the name of the class or package are implemented as one-time hooks, whereas all other such hooks are normal hooks. This allows for the following use case

```
\AddToHook{package/varioref/after}
      { ... apply my customizations if the package gets
        loaded (or was loaded already) ... }
```

without the need to first test if the package is already loaded.

1.4 Hooks for \include files

To manage `\include` files, LaTeX issues a `\clearpage` before and after loading such a file. Depending on the use case one may want to execute code before or after these `\clearpage`s especially for the one that is issued at the end.

Executing code before the final `\clearpage`, means that the code is processed while the last page of the included material is still under construction. Executing code after it means that all floats from inside the include file are placed (which might have added further pages) and the final page has finished.

Because of these different scenarios we offer hooks in three places.[1] None of the hooks are executed when an `\include` file is bypassed because of an `\includeonly` declaration. They are, however, all executed if LaTeX makes an attempt to load the `\include` file (even if it doesn't exist and all that happens is "`No file ⟨filename⟩.tex`").

include/before
include/.../before
include/end
include/.../end
include/after
include/.../after

These are:

include/before, include/⟨*name*⟩/before These hooks are executed (in that order) after the initial `\clearpage` and after `.aux` file is changed to use ⟨*name*⟩`.aux`, but before the ⟨*name*⟩`.tex` file is loaded. In other words they are executed at the very beginning of the first page of the `\include` file.

include/⟨*name*⟩/end, include/end These hooks are executed (in that order) after LaTeX has stopped reading from the `\include` file, but before it has issued a `\clearpage` to output any deferred floats.

include/⟨*name*⟩/after, include/after These hooks are executed (in that order) after LaTeX has issued the `\clearpage` but before is has switched back writing to the main `.aux` file. Thus technically we are still inside the `\include` and if the hooks generate any further typeset material including anything that writes to the `.aux` file, then it would be considered part of the included material and bypassed if it is not loaded because of some `\includeonly` statement.[2]

include/excluded, include/⟨*name*⟩/excluded The above hooks for `\include` files are only executed when the file is loaded (or more exactly the load is attempted). If, however, the `\include` file is explicitly excluded (through an `\includeonly` statement) the above hooks are bypassed and instead the `include/excluded` hook followed by the `include/`⟨*name*⟩`/excluded` hook are executed. This happens after LaTeX has loaded the `.aux` file for this include file, i.e., after LaTeX has updated its counters to pretend that the file was seen.

[1] If you want to execute code before the first `\clearpage` there is no need to use a hook—you can write it directly in front of the `\include`.

[2] For that reason another `\clearpage` is executed after these hooks which normally does nothing, but starts a new page if further material got added this way.

All `include` hooks involving the name of the included file are implemented as one-time hooks (whereas all other such hooks are normal hooks).

If you want to execute code that is run for every `\include` regardless of whether or not it is excluded, use the `cmd/include/before` or `cmd/include/after` hooks.

1.5 High-level interfaces for LaTeX

We do not provide any additional wrappers around the hooks (like filehook or scrlfile do) because we believe that for package writers the high-level commands from the hook management, e.g., `\AddToHook`, etc. are sufficient and in fact easier to work with, given that the hooks have consistent naming conventions.

1.6 Kernel, class, and package interfaces for LaTeX

`\declare@file@substitution` `\declare@file@substitution {⟨file⟩} {⟨replacement-file⟩}`
`\undeclare@file@substitution` `\undeclare@file@substitution {⟨file⟩}`

If ⟨*file*⟩ is requested for loading replace it with ⟨*replacement-file*⟩. `\CurrentFile` remains pointing to ⟨*file*⟩ but `\CurrentFileUsed` will show the file actually loaded.

The main use case for this declaration is to provide a corrected version of a package that can't be changed (due to its license) but no longer functions because of LaTeX kernel changes, for example, or to provide a version that makes use of new kernel functionality while the original package remains available for use with older releases. As such it is mainly meant for use in the LaTeX kernel but other use cases are conceivable.

The `\undeclare@file@substitution` declaration undoes a substitution made earlier.

> *Please do not misuse this functionality and replace a file with another unless if really needed and only if the new version is implementing the same functionality as the original one!*

`\disable@package@load` `\disable@package@load {⟨package⟩} {⟨alternate-code⟩}`
`\reenable@package@load` `\reenable@package@load {⟨package⟩}`

If ⟨*package*⟩ is requested, do not load it but instead run ⟨*alternate-code*⟩ which could issue a warning, error or any other code.

The main use case is for classes that want to restrict the set of supported packages or contain code that make the use of some packages impossible. So rather than waiting until the document breaks they can set up informative messages why certain packages are not available.

The function is only implemented for packages not for arbitrary files and again it should only be applied if there are good reasons for doing this.[3]

1.7 A sample package for structuring the log output

As an application we provide the package structuredlog that adds lines to the `.log` when a file is opened and closed for reading keeping track of nesting level es well. For example, for the current document it adds the lines

[3] Just to be sure: "I don't like this package by somebody else" is not a good one :-)

```
= (LEVEL 1 START) t1lmr.fd
= (LEVEL 1 STOP) t1lmr.fd
= (LEVEL 1 START) supp-pdf.mkii
= (LEVEL 1 STOP) supp-pdf.mkii
= (LEVEL 1 START) nameref.sty
== (LEVEL 2 START) refcount.sty
== (LEVEL 2 STOP) refcount.sty
== (LEVEL 2 START) gettitlestring.sty
== (LEVEL 2 STOP) gettitlestring.sty
= (LEVEL 1 STOP) nameref.sty
= (LEVEL 1 START) ltfilehook-doc.out
= (LEVEL 1 STOP) ltfilehook-doc.out
= (LEVEL 1 START) ltfilehook-doc.out
= (LEVEL 1 STOP) ltfilehook-doc.out
= (LEVEL 1 START) ltfilehook-doc.hd
= (LEVEL 1 STOP) ltfilehook-doc.hd
= (LEVEL 1 START) ltfilehook.dtx
== (LEVEL 2 START) ot1lmr.fd
== (LEVEL 2 STOP) ot1lmr.fd
== (LEVEL 2 START) omllmm.fd
== (LEVEL 2 STOP) omllmm.fd
== (LEVEL 2 START) omslmsy.fd
== (LEVEL 2 STOP) omslmsy.fd
== (LEVEL 2 START) omxlmex.fd
== (LEVEL 2 STOP) omxlmex.fd
== (LEVEL 2 START) umsa.fd
== (LEVEL 2 STOP) umsa.fd
== (LEVEL 2 START) umsb.fd
== (LEVEL 2 STOP) umsb.fd
== (LEVEL 2 START) ts1lmr.fd
== (LEVEL 2 STOP) ts1lmr.fd
== (LEVEL 2 START) t1lmss.fd
== (LEVEL 2 STOP) t1lmss.fd
= (LEVEL 1 STOP) ltfilehook.dtx
```

Thus if you inspect an issue in the `.log` it is easy to figure out in which file it occurred, simply by searching back for `LEVEL` and if it is a `STOP` then remove 1 from the level value and search further for `LEVEL` with that value which should then be the `START` level of the file you are in.

Index

The italic numbers denote the pages where the corresponding entry is described, numbers underlined point to the definition, all others indicate the places where it is used.

Symbols

/after (hook) 2, 2, 3
/before (hook) 2, 2

A

\AddToHook 5
\AtEndOfClass 3
\AtEndOfPackage 3

C

class/ (hook) 3
class/.../after 3
class/.../before 3
class/⟨*name*⟩/after (hook) 3
class/⟨*name*⟩/before (hook) 3
class/after (hook) 3
class/after 3
class/before (hook) 3
class/before 3
\clearpage 4
cmd/include/after (hook) 5
cmd/include/before (hook) 5
\CurrentFile 3
\CurrentFilePath 3
\CurrentFilePathUsed 3
\CurrentFileUsed 3

F

file commands:
  \l_file_search_path_seq 2
file/.../after 2
file/.../before 2
file/⟨*file name*⟩/after (hook) 2, 2
file/⟨*file name*⟩/before (hook) 2, 2
file/⟨*file-name*⟩/after (hook) 2
file/⟨*file-name*⟩/before (hook) 2
file/⟨*package name*⟩.sty/after (hook) . 3
file/⟨*package name*⟩.sty/before (hook) 3
file/after (hook) 2, 2, 3
file/after 2
file/array.sty/after (hook) 3
file/before (hook) 2, 3, 2
file/before 2

H

Hooks:
  /after 2, 2, 3
  /before 2, 2
  class/ 3
  class/⟨*name*⟩/after 3
  class/⟨*name*⟩/before 3
  class/after 3
  class/before 3
  cmd/include/after 5
  cmd/include/before 5
  file/⟨*file name*⟩/after 2, 2
  file/⟨*file name*⟩/before 2, 2
  file/⟨*file-name*⟩/after 2
  file/⟨*file-name*⟩/before 2
  file/⟨*package name*⟩.sty/after 3
  file/⟨*package name*⟩.sty/before . . . 3
  file/after 2, 2, 3
  file/array.sty/after 3
  file/before 2, 3, 2
  include 5
  include/⟨*name*⟩/after 4
  include/⟨*name*⟩/before 4
  include/⟨*name*⟩/end 4
  include/⟨*name*⟩/excluded 4, 4
  include/after 4
  include/before 4
  include/end 4
  include/excluded 4, 4
  package/ 3
  package/⟨*name*⟩/after 3
  package/⟨*name*⟩/before 3
  package/⟨*package name*⟩/after 3
  package/⟨*package name*⟩/before 3
  package/after 3, 3
  package/before 3, 3

I

include (hook) 5
\include 1
include/.../after 4
include/.../before 4
include/.../end 4
include/⟨*name*⟩/after (hook) 4
include/⟨*name*⟩/before (hook) 4
include/⟨*name*⟩/end (hook) 4
include/⟨*name*⟩/excluded (hook) 4, 4
include/after (hook) 4
include/after 4
include/before (hook) 4
include/before 4
include/end (hook) 4
include/end 4

include/excluded (hook) 4, 4
\includeonly 4
\input 2

L

\LoadPackageWithOptions 3

O

\openin 1

P

package/ (hook) 3
package/.../after 3
package/.../before 3
package/⟨*name*⟩/after (hook) 3
package/⟨*name*⟩/before (hook) 3
package/⟨*package name*⟩/after (hook) . . 3
package/⟨*package name*⟩/before (hook) . 3
package/after (hook) 3, 3
package/after 3
package/before (hook) 3, 3
package/before 3

R

\RequirePackage 3

T

TEX and LATEX 2ε commands:

\@input 1
\declare@file@substitution 5
\disable@package@load 5
\input@path 2
\reenable@package@load 5
\undeclare@file@substitution 5

U

\UseHook 3
\UseOneTimeHook 3
\usepackage 3